

SpirulinaAI — Deployment Handoff

For whoever is standing this up on a server. This documents what the app needs to run: services, databases, model files, env vars, and — important — several things that only exist on the original dev machine right now and need a decision before this can go live elsewhere.

0. Status — what's done vs. what's still needed before this is public

✓ Already fixed (2026-07-27)

- **Auth is now actually enforced.** Every route that touches user or container data (`/chat` , `/conversations/*` , `/alerts/*` , `/sensors/*` , `/models/*` , `/cpc/predict` , `/species/predict`) now requires a valid login token, and the server trusts the token's identity over anything the client sends in the request body — a request can't chat as, or read the conversations of, a different `user_id` than the one it's logged in as (verified with a real signup → real JWT → cross-user access → 403 test, not just a code review). `/status` and the manual `/debug/run-combination-check/{container_id}` endpoint are now admin-only, since they were dev-only tools sitting open before.
- **Upload endpoints have a size limit** — `/cpc/predict` and `/species/predict` reject files over 10MB, closing an anonymous compute-abuse angle.
- **The two ML models that lived outside the repo are now bundled in it** (§3) — `git clone` gets everything, no more copying files off someone's `G:\` drive.
- **CORS is now configurable** via `CORS_ALLOWED_ORIGINS` — see the item below, this made it possible to lock down, it didn't lock it down itself.

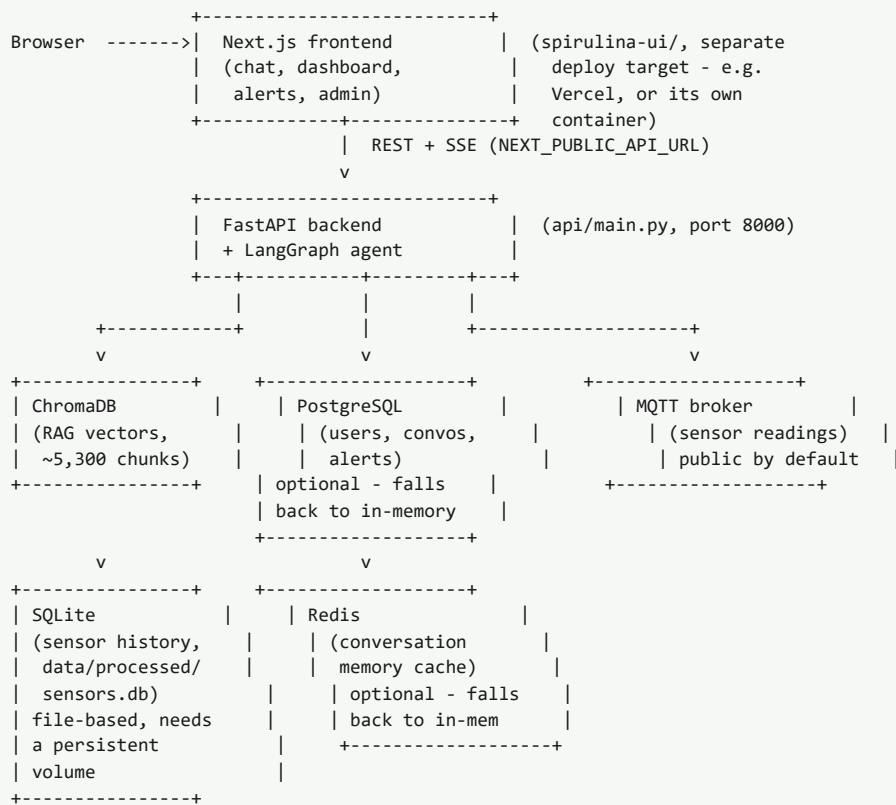
⚠ Still needed before going live — these are yours to do, not code fixes

1. `JWT_SECRET` is still the insecure default in your actual `.env` right now (`spirulina-secret-key-change-in-production`) — confirmed by checking the file directly, this isn't just a theoretical warning. Anyone who reads this codebase can forge a login token for any user, including admin, until this is changed. Generate a real random secret (e.g. `openssl rand -hex 32`) and set it in production's `.env` .
2. Set `CORS_ALLOWED_ORIGINS` to your friend's actual frontend domain once deployed. It still defaults to `"*"` (unchanged from before, so nothing broke locally) — that default is fine for testing, not for a public launch.
3. `DATABASE_URL` currently points at `localhost:5433` , which won't be reachable from wherever this actually deploys. Without a real reachable Postgres, conversation history and the free-tier message cap silently stop working (falls back to in-memory — confirmed by testing this directly).
4. The checked-in `Dockerfile` is still stale — it predates auth, the MQTT pipeline, and all three ML models. As-is, `docker build` produces a container that crashes on startup (it never copies `models/`). Needs updating before it's used — see §7. Not yet fixed.
5. The LLM currently runs locally via Ollama (`qwen3:8b`) on the dev machine's GPU. A cloud VM without a GPU will run this very slowly on CPU. Decide up front whether to self-host an LLM (needs a

GPU instance) or switch back to a hosted API (Groq/OpenRouter — the code already supports both, it's an env var flip). See §4.

6. **The MQTT broker is the public** `test.mosquitto.org` — fine for testing, not recommended for real sensor data long-term (no auth, no privacy, shared with anyone else using that broker).
7. `.env.template` **currently contains a real (if free-tier) Groq API key**, not a placeholder. It's git-ignored so it was never pushed to any repo, but **don't hand this specific file to anyone as-is** — strip the key out (or rotate it) before sharing.

1. Architecture overview



LLM inference (3 roles - intent router, RAG generator, reasoning agent):
either a local Ollama server (qwen3:8b, needs GPU for reasonable latency) or hosted APIs (Groq / OpenRouter) - provider is an env var per role, see section 4.

3 ML models loaded in-process by the FastAPI backend:

- `models/anomaly_model/` - self-contained, artifacts already in git
- `models/cpc_model/` - model file is OUTSIDE the repo, see section 3
- `models/species_classifier/` - model files OUTSIDE the repo, see section 3

2. Services checklist

Service	Required?	Notes
FastAPI backend	Required	<code>api/main.py</code> , Python (repo currently pins Docker to 3.10; dev machine now runs 3.13 — pick one, see §7)

Next.js frontend	Required	<code>spirulina-ui/</code> , deployed separately from the backend
ChromaDB	Required	Embedded/file-based, not a separate server — just needs its persist directory to survive restarts
SQLite (sensors)	Required	Also just a file (<code>data/processed/sensors.db</code>) — needs a persistent volume, same as Chroma
PostgreSQL	Optional, but strongly recommended	Without it: no persistent user accounts/conversations/alerts across restarts, and the free-tier 3-msg/day cap silently doesn't apply
Redis	Optional	Without it: conversation memory is process-local and lost on every backend restart
MQTT broker	Required for live sensor data	Defaults to the public <code>test.mosquitto.org</code> — fine for testing, not recommended for real production data (no auth, no privacy, anyone on that broker can see your topics). Consider a private broker (e.g. self-hosted Mosquitto, HiveMQ Cloud, EMQX Cloud) for a real deployment.
Ollama (local LLM)	Optional (one of two choices)	Only needed if self-hosting the LLM — see §4
Sentry	Optional	Wired in, inactive until <code>SENTRY_DSN</code> is set
Logstash	Optional	Wired in, inactive until <code>LOGSTASH_HOST</code> is set
AWS Cognito	Optional	Wired in, inactive until <code>COGNITO_*</code> vars are set — custom JWT auth works fine without it

3. Model files — now bundled in the repo

All three models are now self-contained — `git clone` gets everything.

Model	Location in repo	Size
Anomaly detector (<code>models/anomaly_model/</code>)	<code>artifacts/combination_model.joblib</code> + <code>artifacts/seasonal_baseline.json</code>	~140 KB
CPC / biomass predictor (<code>models/cpc_model/</code>)	<code>artifacts/biomass_cpc_model_random_split.pt</code>	~1.9 MB
Species classifier (<code>models/species_classifier/</code>)	<code>artifacts/xgboost_model.joblib</code> , <code>scaler.joblib</code> , <code>label_encoder.joblib</code> , <code>feature_names.joblib</code> , <code>raw_feature_stats.joblib</code>	~1.9 MB

Both predictors' `_DEFAULT_PATH` / `_DEFAULT_DIR` now point at their own `artifacts/` folder (`Path(__file__).parent / "artifacts"`) instead of a Windows-only absolute path. `CPC_MODEL_PATH` and `SPECIES_MODEL_DIR` still work as **optional** overrides if your friend ever wants to swap in a retrained checkpoint without touching code — they just aren't required anymore. Verified working: both predictors load and run inference straight from a fresh clone with no env vars set.

4. LLM — local vs. hosted (decision needed)

The app has **three LLM roles** (intent router, RAG generator, reasoning agent), each independently configurable via a `*_MODEL_PROVIDER` env var. Right now all three are set to `ollama` on the dev machine.

Option	Pros	Cons
A. Keep Ollama, self-hosted	Free, private, no per-token cost, no rate limit	Needs a GPU on the server for usable latency (the dev machine uses an 8GB-VRAM laptop GPU for <code>qwen3:8b</code>) — a plain CPU cloud VM will be slow (seconds-to-tens-of-seconds per response); Ollama itself must be installed and running as its own service, <code>qwen3:8b</code> pulled (~5GB download)
B. Switch to Groq (hosted API)	Fast, no GPU needed, was the original setup, code already supports it	Free tier has a daily token quota (the reason the project switched away from it) — fine for light/demo traffic, will need a paid Groq plan for real usage
C. Mix	e.g. Groq for the fast intent router, keep something else for reasoning	More moving parts, more to configure

To switch back to Groq: set `INTENT_MODEL_PROVIDER=groq`, `GENERATOR_MODEL_PROVIDER=groq`, `REASONING_MODEL_PROVIDER=groq` (or `openrouter` for the reasoning role, as the project used before), and supply `GROQ_API_KEY` (and `OPENROUTER_API_KEY` if using OpenRouter). No code changes needed — this is exactly what the env vars are for.

If keeping Ollama: the server needs Ollama installed (ollama.com), running (`ollama serve`), and the model pulled (`ollama pull qwen3:8b`) *before* the backend starts, with `OLLAMA_BASE_URL` pointing at it (`http://localhost:11434` if same host, or a network address if Ollama runs on a separate GPU box).

5. Embeddings (RAG)

- `EMBED_MODEL` env var controls this — `.env.template` currently says `BAAI/bge-m3` (1024-dim, multilingual), but **one prior observed run actually loaded a different model** (`paraphrase-multilingual-mpnet-base-v2`) instead. **Confirm which one is actually in effect before deploying** — check the live `EMBED_MODEL` value and, if in doubt, just re-run ingestion fresh so the ChromaDB collection and the configured model are guaranteed to match (see next point).
- The embedding model is downloaded from Hugging Face on first use** (not bundled in the repo or image) — a few hundred MB to ~2GB depending on which model. `docker-compose.yml` already mounts a `model_cache` volume (`/root/.cache/torch`) so this only happens once per environment, not on every container restart — make sure that volume (or equivalent persistent cache) exists on whatever platform this deploys to.
- Changing `EMBED_MODEL` requires deleting the existing ChromaDB directory and re-running `python -m rag.embedder.ingest`** — the vector dimensions won't match otherwise and it'll crash, not silently degrade.
- The ChromaDB data itself (`data/processed/chroma/` , ~20 MB, ~5,300 chunks from 20+ documents) **is already committed to git** — no re-ingestion needed for a first deploy, only if `EMBED_MODEL` changes.

6. Full environment variable reference

Pulled directly from every `os.getenv( ... )` call in the codebase — this is the real list, not just what's in `.env.template` (which is missing a few of these).

Required for a real deployment

Var	Purpose	Default if unset
-----	---------	------------------

<code>JWT_SECRET</code>	Signs auth tokens	Insecure hardcoded fallback — must override
<code>API_HOST</code> / <code>API_PORT</code>	FastAPI bind address	<code>0.0.0.0</code> / <code>8000</code>
<code>CORS_ALLOWED_ORIGINS</code>	Comma-separated list of allowed frontend origins (e.g. <code>https://app.example.com</code>)	<code>"*"</code> (wide open) if unset — fine for testing, lock this down for a public launch

LLM / RAG

Var	Purpose	Notes
<code>INTENT_MODEL_PROVIDER</code> , <code>INTENT_MODEL_NAME</code>	Intent router	<code>groq</code> / <code>openai</code> / <code>ollama</code>
<code>GENERATOR_MODEL_PROVIDER</code> , <code>GENERATOR_MODEL_NAME</code>	RAG answer generation	<code>groq</code> / <code>openai</code> / <code>huggingface</code> / <code>ollama</code>
<code>REASONING_MODEL_PROVIDER</code> , <code>REASONING_MODEL_NAME</code>	Harvest/system reasoning agent	defaults to generator's provider if unset
<code>OLLAMA_BASE_URL</code>	Ollama server address	only used when a provider above is <code>ollama</code>
<code>GROQ_API_KEY</code>	Groq API auth	needed if any provider is <code>groq</code>
<code>OPENROUTER_API_KEY</code>	OpenRouter API auth	needed if reasoning provider is <code>openrouter</code>
<code>EMBED_MODEL</code>	Sentence-transformers model for RAG	see §5
<code>CHROMA_PERSIST_DIR</code>	ChromaDB storage path	<code>./data/processed/chroma</code>
<code>RAW_DATA_DIR</code>	Source docs for ingestion (one-time)	<code>data/raw</code>

Databases / storage

Var	Purpose	Notes
<code>DATABASE_URL</code>	PostgreSQL connection string	<code>postgresql://user:pass@host:5432/dbname</code> — leave empty for in-memory fallback (no persistence, no free-tier enforcement)
<code>REDIS_URL</code>	Redis connection string	e.g. <code>redis://localhost:6379</code> — leave empty for in-memory fallback

Sensors / monitoring

Var	Purpose	Default
<code>MQTT_BROKER_URL</code>	MQTT broker address	<code>mqtt://localhost:1883</code> in code, but the dev deployment actually points at the public <code>test.mosquitto.org:1883</code> — set this explicitly
<code>MQTT_TOPIC_PREFIX</code>	Topic namespace	check <code>agent/sensors.py</code> for current value

<code>MONITOR_RULE_INTERVAL_SECONDS</code>	How often the anomaly/threshold check runs	<code>900</code> (15 min) — matches real MQTT cadence; the dev machine overrides this to <code>15</code> for fast local demoing, don't ship that override to production
--	--	---

ML models (both optional now — see §3)

Var	Purpose	Default
<code>CPC_MODEL_PATH</code>	Override the bundled CPC/biomass checkpoint	<code>models/cpc_model/artifacts/biomass_cpc_model_random_split.pt</code> (in-repo)
<code>SPECIES_MODEL_DIR</code>	Override the bundled species-classifier folder	<code>models/species_classifier/artifacts/</code> (in-repo)

Auth (optional — Cognito)

Var	Purpose
<code>COGNITO_REGION</code> , <code>COGNITO_USER_POOL_ID</code> , <code>COGNITO_APP_CLIENT_ID</code>	Activates AWS Cognito RS256 token verification alongside custom JWT. Leave blank to use custom JWT only (works fine standalone).

Observability (optional)

Var	Purpose
<code>SENTRY_DSN</code>	Enables Sentry error tracking
<code>SENTRY_TRACES_SAMPLE_RATE</code>	Sentry performance sampling
<code>LOGSTASH_HOST</code> , <code>LOGSTASH_PORT</code>	Enables log shipping to Logstash
<code>LOG_LEVEL</code>	Python logging level, default <code>INFO</code>
<code>ENVIRONMENT</code> , <code>APP_VERSION</code>	Tags on logs/Sentry events

Frontend (`spirulina-ui/.env.local` — separate from the backend's `.env`)

Var	Purpose
<code>NEXT_PUBLIC_API_URL</code>	Backend base URL the frontend calls — currently <code>http://localhost:8000</code> , must point at the deployed backend's public URL/domain

7. Docker setup — current state and what needs fixing

`Dockerfile` + `docker-compose.yml` exist but are **out of date** relative to the current app:

- Copies only `agent/`, `api/`, `rag/`, `data/raw/`, `chat.html` — **does not copy** `models/` (anomaly detector, CPC, species classifier) or the root-level `data/processed/` directory. As committed, the image will fail to import at startup.
- Base image is `python:3.10-slim`; the project's actual dev environment moved to Python 3.13 months ago. Not necessarily broken (3.10 should still satisfy `requirements.txt`), but untested against the

current dependency set — worth a clean build-and-boot test before relying on it.

- No Ollama service in `docker-compose.yml` (only `redis` + `backend`) — expected, since Ollama wasn't part of the stack when this compose file was written; needs adding if going with Option A in §4, or removing the Ollama env vars if going with Option B.
- No frontend service/Dockerfile at all — `spirulina-ui/` needs its own deploy path (its own container, or a platform like Vercel/Netlify built for Next.js).
- `data/processed/sensors.db` isn't mounted as a volume anywhere in `docker-compose.yml` (only `chroma_data` and `model_cache` are) — add one, or every container restart loses sensor history.

Recommended before handoff: update the `Dockerfile`'s `COPY` lines to include `models/` and `data/processed/`, add a volume for the sensors SQLite file, and do one full `docker compose up --build` locally to confirm the container actually boots and answers `GET /health` before your friend inherits it.

8. Checklist for your friend

1. ~~Get the 5 external model files~~ Done — already bundled in the repo (§3), nothing to do here.
2. ~~Wire up auth enforcement~~ Done — see §0, verified with real signup/login/cross-user tests.
3. Decide LLM hosting: self-hosted Ollama (needs GPU) vs. Groq/OpenRouter API (needs API keys, has quota limits on free tier) — §4.
4. Set `JWT_SECRET` to a real random secret. Don't ship the default — still the literal default in the dev `.env` as of this writing.
5. Set `CORS_ALLOWED_ORIGINS` to the real deployed frontend domain once it has one. Still `"*"` by default.
6. Get a fresh Groq API key if going with hosted LLMs — don't reuse the one sitting in the local `.env.template`; rotate/replace it.
7. Stand up Postgres + Redis (or accept the in-memory fallbacks for a quick demo deploy, with the tradeoffs in §2) — `DATABASE_URL` currently points at `localhost:5433`, which won't resolve anywhere but the dev machine.
8. Decide on an MQTT broker — public `test.mosquitto.org` for a quick test, a private broker for anything real.
9. Fix the `Dockerfile` / `docker-compose.yml` gaps in §7, or containerize from scratch if that's easier. Not yet fixed.
10. Deploy the backend, confirm `EMBED_MODEL` matches what the committed ChromaDB was actually built with (§5) before trusting RAG answers.
11. Deploy `spirulina-ui/` separately, point `NEXT_PUBLIC_API_URL` at the live backend.
12. Persist volumes for: ChromaDB dir, sensors SQLite file, Ollama model cache (if self-hosting), HF embedding-model cache.

Compiled by reading every `os.getenv()` call, both model predictors' default paths, `Dockerfile` / `docker-compose.yml`, `.env.template`, and `data/store.py` directly from the current source tree.